

Kubernetes RBAC Made Simple: Who Can Do What

Kubernetes Role-Based Access Control (RBAC) controls who can access your cluster and what actions they can perform. Understanding RBAC helps you prevent accidental cluster deletions and keep your production applications secure.

What you'll learn:

- 1 Creating Namespace Roles
- 2 Binding Roles to Users
- 3 Creating Service Accounts for Apps
- 4 Testing Access with 'can-i'

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Creating Namespace Roles

A Role defines what actions are allowed on resources inside one specific namespace. Think of it like giving someone a key card that opens only one department door. You specify allowed actions using verbs like get, list, or delete on target resources.

```
$kubectl create role pod-manager --verb=get,list,create,delete --resource=pods -n dev
```

Cloud & DevOps Daily

Kubernetes RBAC: Who Can Do What in Your Cluster

Friday, September 25, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Binding Roles to Users

A Role does nothing until you connect it to an identity using a RoleBinding. Think of a Role as a set of key permissions and a RoleBinding as handing that key to a real user or bot. Without a binding, the user remains blocked.

```
$kubectl create rolebinding dev-pod-manager --role=pod-manager --user=sarah -n dev
```

Cloud & DevOps Daily

Kubernetes RBAC: Who Can Do What in Your Cluster

Friday, September 25, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Creating Service Accounts for Apps

Human users log in with user accounts, but applications running in pods need ServiceAccounts. When an application needs to talk to the Kubernetes API, you attach a ServiceAccount to its deployment so it gets the correct privileges.

```
$kubectl create serviceaccount pipeline-runner -n dev
```

Cloud & DevOps Daily

Kubernetes RBAC: Who Can Do What in Your Cluster

Friday, September 25, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Testing Access with 'can-i'

Before giving permissions to a teammate or service account, test what they can actually do. The 'auth can-i' command asks the API server if an action is allowed and returns a simple 'yes' or 'no' response.

```
$kubectl auth can-i delete pods --as=system:serviceaccount:dev:pipeline-runner -n dev
```

Cloud & DevOps Daily

Kubernetes RBAC: Who Can Do What in Your Cluster

Friday, September 25, 2026

See Key Takeaways on next page >> linkedin.com/in/mrmanrajsingh

Key Takeaways

- + Always start with empty permissions and add only what is necessary.
- + Use Roles over ClusterRoles whenever access is only needed in one namespace.
- + Assign ServiceAccounts to pods instead of embedding admin credentials inside app code.
- + Test permissions using 'kubectl auth can-i' before giving access to users.
- + Group team members into role-based groups instead of binding permissions user by user.

Watch Out For

- ! Granting cluster-admin access to resolve permission errors quickly during testing.
- ! Forgetting to specify the namespace parameter (-n) when creating Roles or RoleBindings.
- ! Using human user accounts for automated CI/CD tools instead of dedicated ServiceAccounts.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh